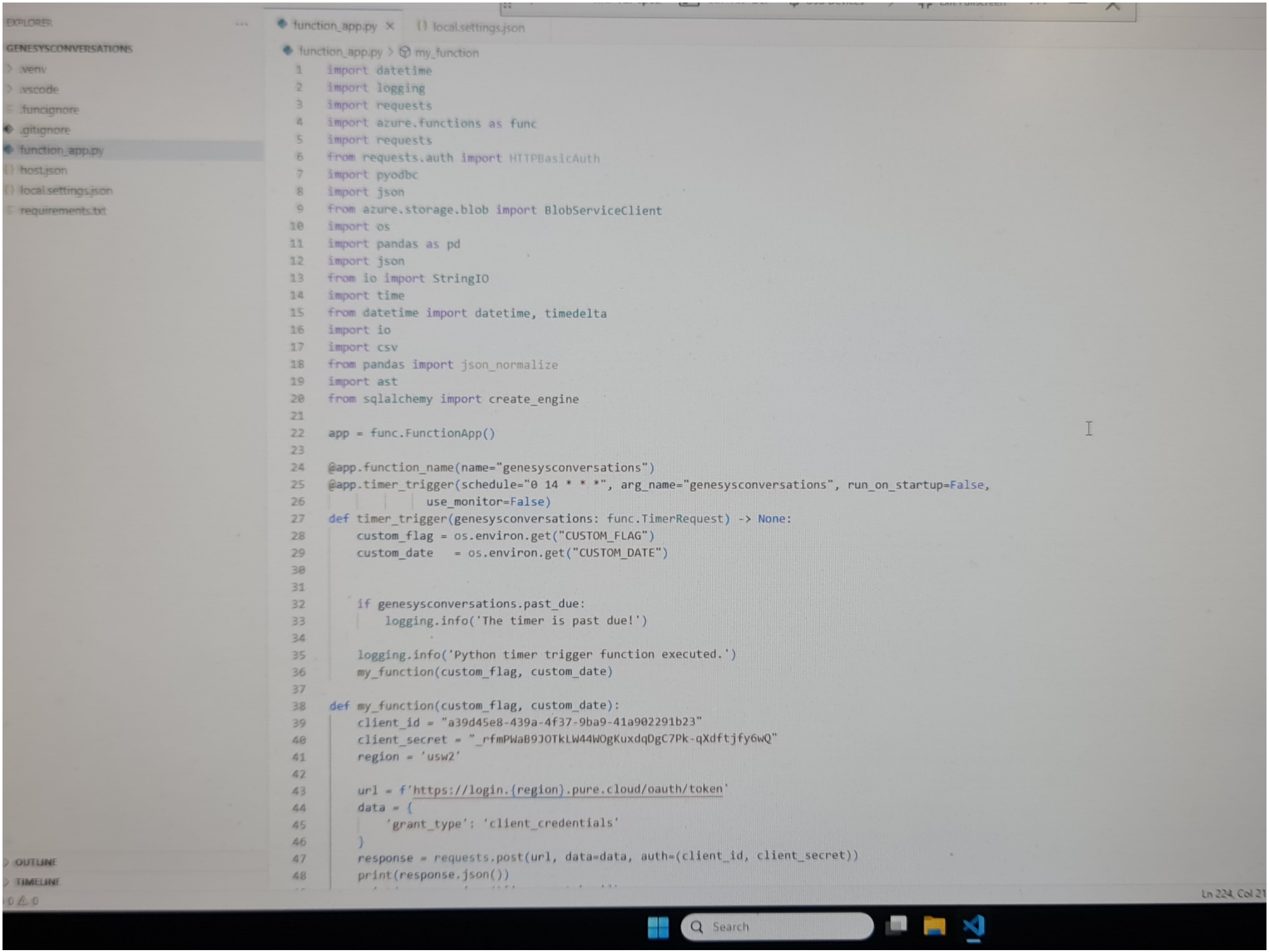

EXPLORER
GENESYSCONVERSATIONS
.venv
.vscode
.funcignore
.gitignore
function_app.py
host.json
local.settings.json
requirements.txt
function_app.py
local.settings.json
function_app.py > my_function
import datetime
import logging
import requests
import azure.functions as func
import requests
from requests.auth import HTTPBasicAuth
import pyodbc
import json
from azure.storage.blob import BlobServiceClient
import os
import pandas as pd
import json
from io import StringIO
import time
from datetime import datetime, timedelta
import io
import csv
from pandas import json_normalize
import ast
from sqlalchemy import create_engine
app = func.FunctionApp()
@app.function_name(name="genesysconversations")
@app.timer_trigger(schedule="0 14 * * *", arg_name="genesysconversations", run_on_startup=False,
use_monitor=False)
def timer_trigger(genesysconversations: func.TimerRequest) -> None:
custom_flag = os.environ.get("CUSTOM_FLAG")
custom_date = os.environ.get("CUSTOM_DATE")
if genesysconversations.past_due:
logging.info('The timer is past due!')
logging.info('Python timer trigger function executed.')
my_function(custom_flag, custom_date)
def my_function(custom_flag, custom_date):
client_id = "a39d45e8-439a-4f37-9ba9-41a902291b23"
client_secret = "_rfmPWaB9JOTkLW44WOgKuxdqDgC7Pk-qXdftjfy6wQ"
region = 'usw2'
url = f'https://login.{region}.pure.cloud/oauth/token'
data = {
'grant_type': 'client_credentials'
}
response = requests.post(url, data=data, auth=(client_id, client_secret))
print(response.json())
OUTLINE
TIMELINE
Ln 224, Col 21
Search

Selection View Go Run Terminal Help

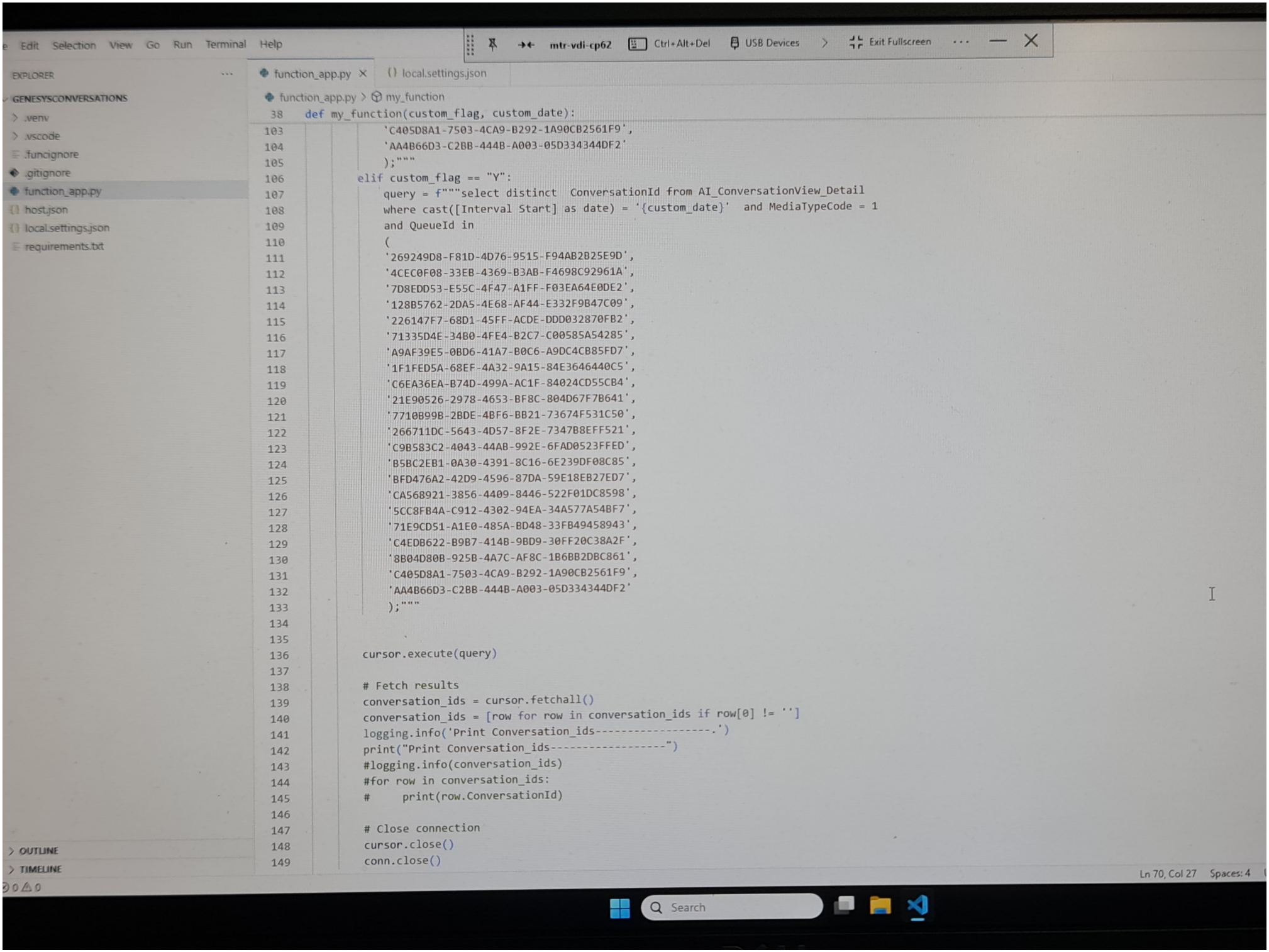
mtr-vdi-cp62 Ctrl+Alt+Del USB Devices Exit Fullscreen

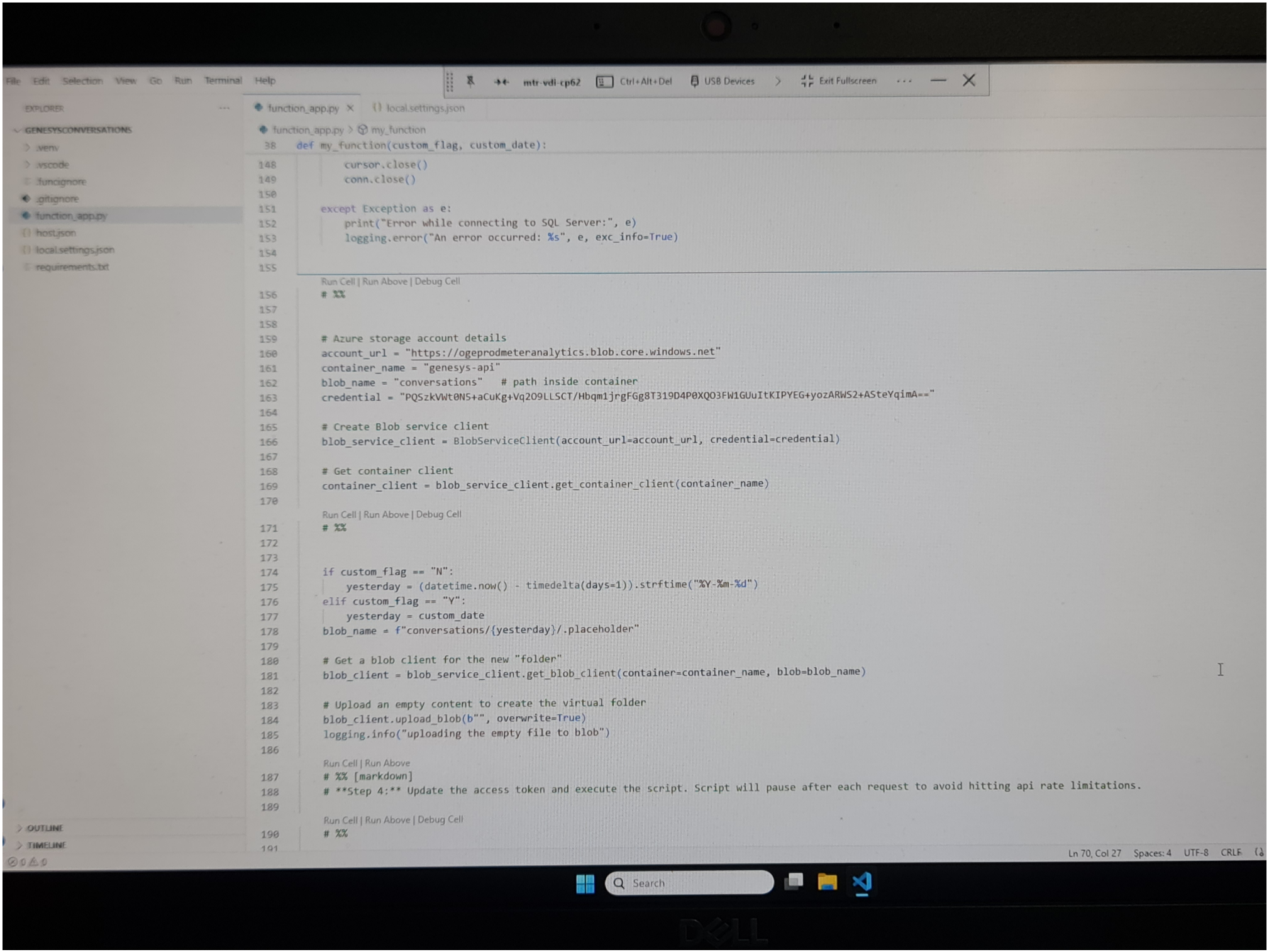
function_app.py local.settings.json

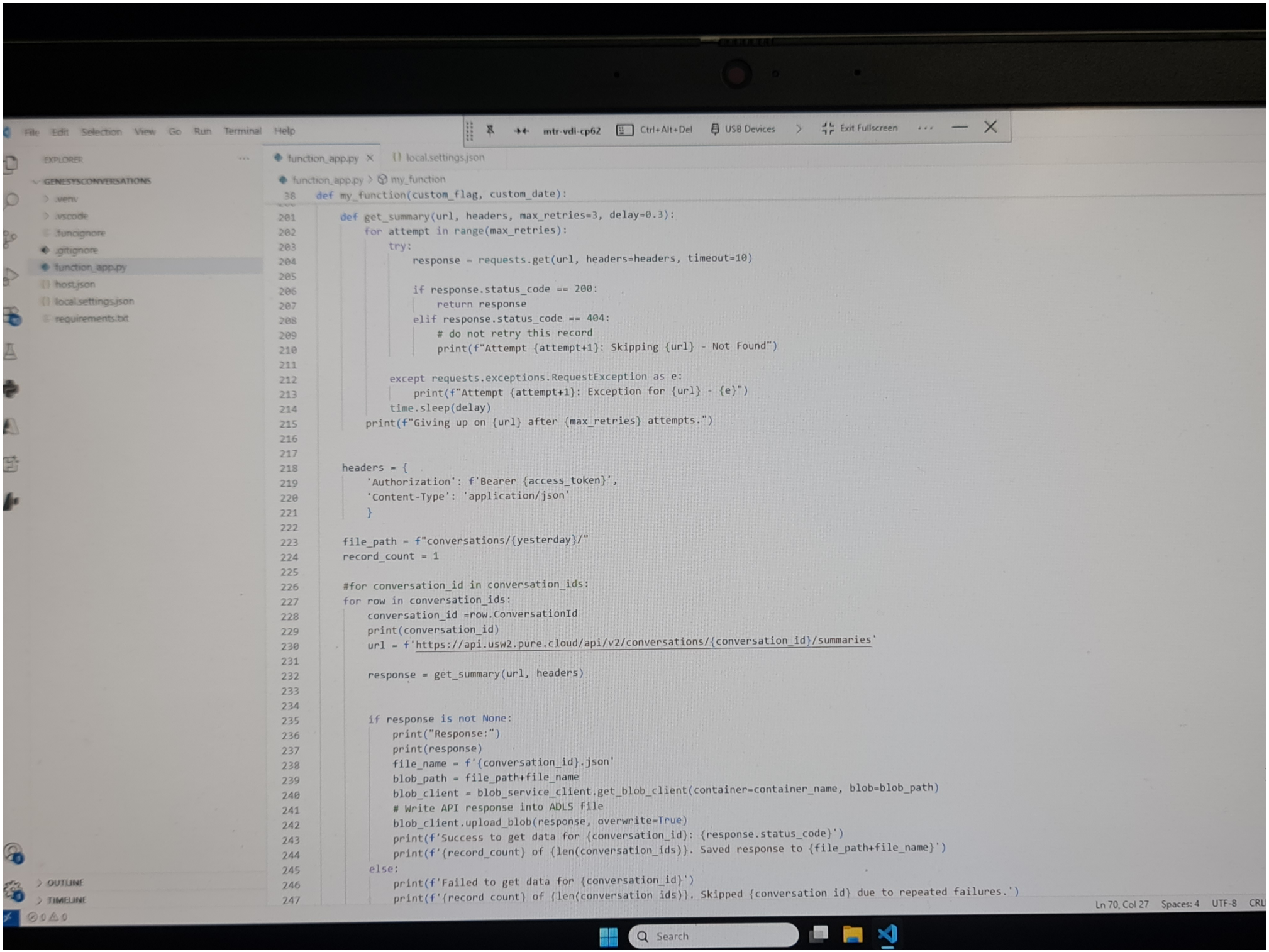
function_app.py > my_function

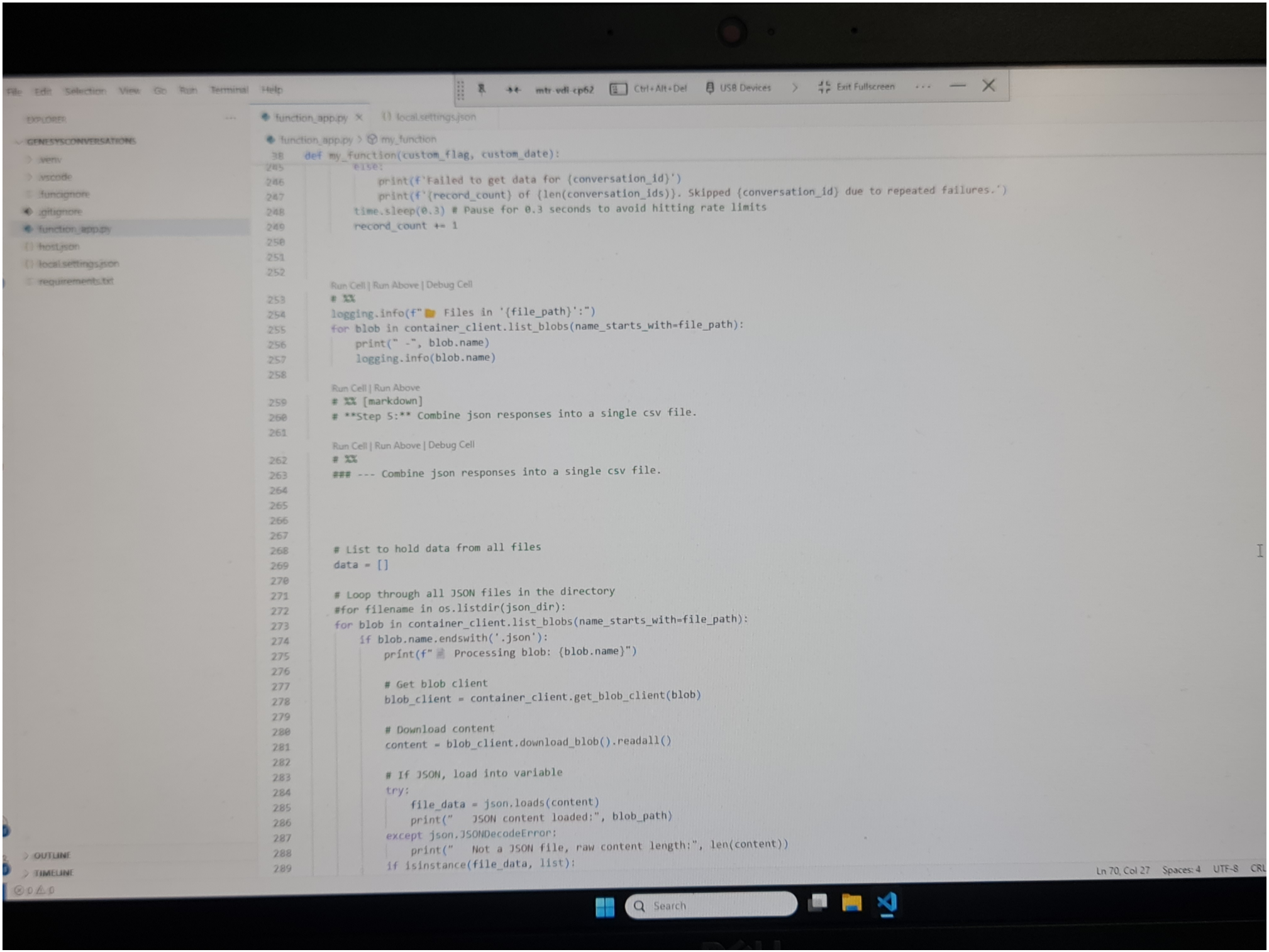
```
38  def my_function(custom_flag, custom_date):
47      response = requests.post(url, data=data, auth=(client_id, client_secret))
48      print(response.json())
49      print(response.json()['access_token'])
50      access_token = response.json()['access_token']
51      logging.info(access_token)
52
        Run Cell | Run Below
53      # %% [markdown]
54      # **Step 3:** Get list of Conversation IDs from SQL Server
55
        Run Cell | Run Above | Debug Cell
56      # %%
57
58
59      # Connection details
60      server = 'insights-westcentral-sqlserver.database.windows.net'
61      database = '1080-OGE-Production'
62      username = 'OGEProductionreadonly'
63      password = 'Pr^HF#ZV'
64
65      conn_str = (
66          "DRIVER={ODBC Driver 18 for SQL Server};"
67          f"SERVER={server};"
68          f"DATABASE={database};"
69          f"UID={username};"
70          f"PWD={password};"
71      )
72
73      try:
74          # Connect to SQL Server
75          conn = pyodbc.connect(conn_str)
76          cursor = conn.cursor()
77
78          if custom_flag == "N":
79              query = """select distinct  ConversationId from AI_ConversationView_Detail
80              where cast([Interval Start] as date) = cast(getdate()-1 as date)  and MediaTypeCode = 1
81              and QueueId in
82              (
83              '269249D8-F81D-4D76-9515-F94AB2B25E9D',
84              '4CEC0F08-33EB-4369-B3AB-F4698C92961A',
85              '7D8EDD53-E55C-4F47-A1FF-F03EA64E0DE2',
86              '128B5762-2DA5-4E68-AF44-E332F9B47C09',
87              '226147F7-68D1-45FF-ACDE-DDD032870FB2',
88              '71335D4E-34B0-4FE4-B2C7-C00585A54285',
89              'A9AF39E5-0BD6-41A7-B0C6-A9DC4CB85FD7',
90              '1F1FED5A-68EF-4A32-9A15-84E3646440C5',
91              'C6EA36EA-B74D-499A-AC1F-84024CD55CB4',
```

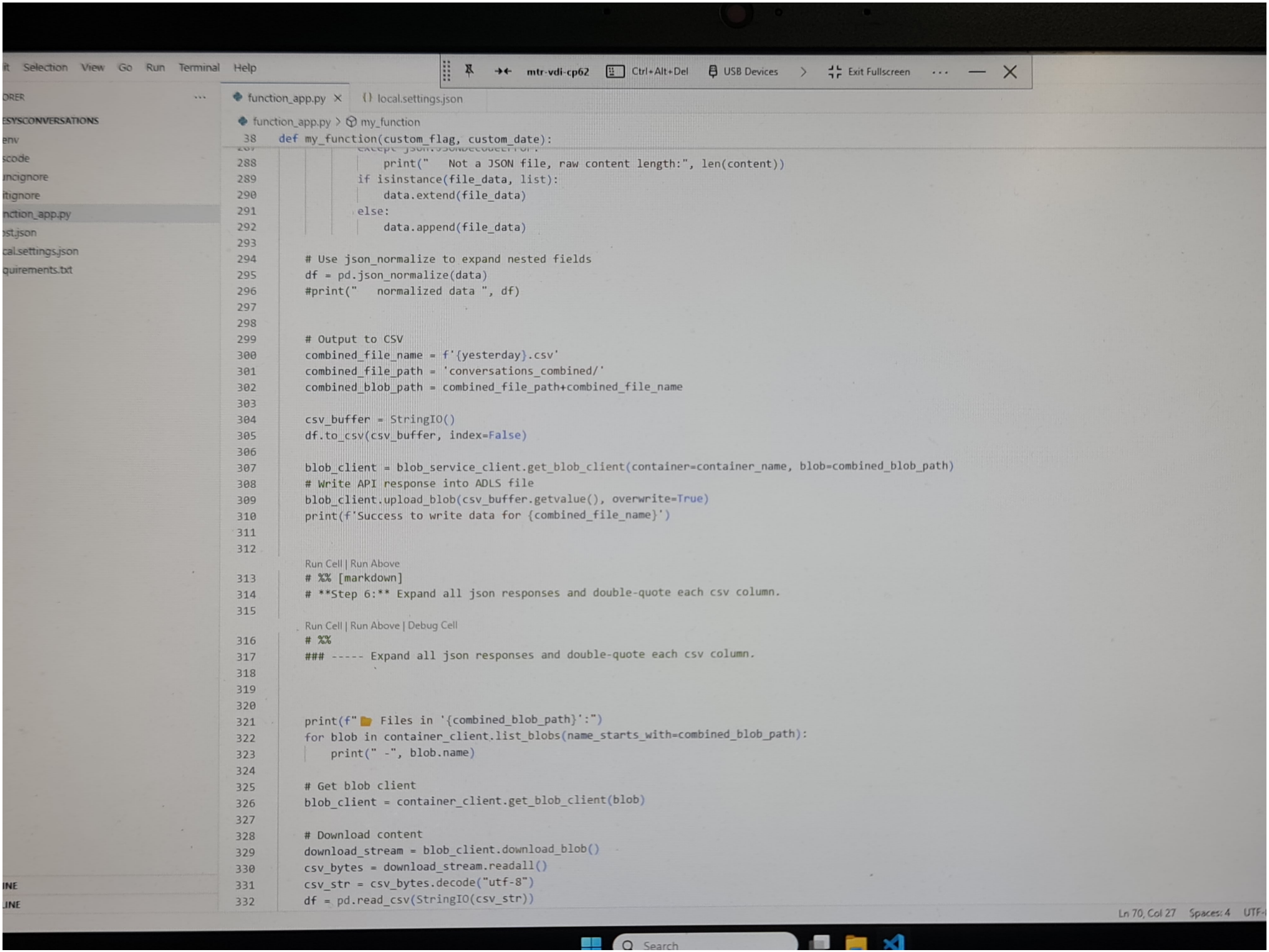
Ln 70, Col 27

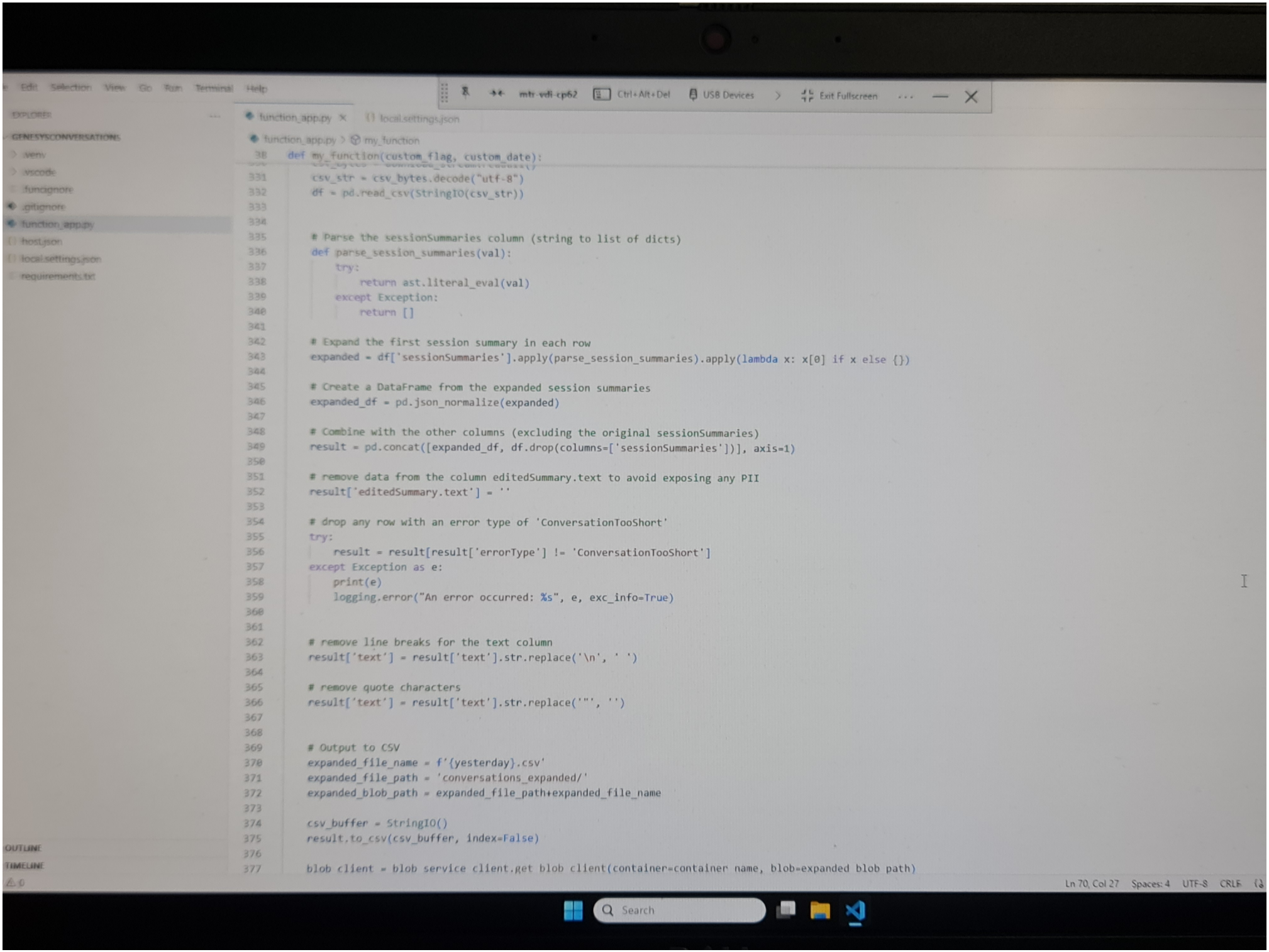

Edit Selection View Go Run Terminal Help
mtr-vdi-cp62 Ctrl+Alt+Del USB Devices Exit Fullscreen
EXPLORER
function_app.py
local.settings.json
GENESYSCONVERSATIONS
.venv
.vscode
.funcignore
.gitignore
function_app.py
host.json
local.settings.json
requirements.txt
function_app.py > my_function
def my_function(custom_flag, custom_date):
'C405D8A1-7503-4CA9-B292-1A90CB2561F9',
'AA4B66D3-C2BB-444B-A003-05D334344DF2'
);"""
elif custom_flag == "Y":
query = f"""select distinct ConversationId from AI_ConversationView_Detail
where cast([Interval Start] as date) = '{custom_date}' and MediaTypeCode = 1
and QueueId in
(
'269249D8-F81D-4D76-9515-F94AB2B25E9D',
'4CEC0F08-33EB-4369-B3AB-F4698C92961A',
'7D8EDD53-E55C-4F47-A1FF-F03EA64E0DE2',
'128B5762-2DA5-4E68-AF44-E332F9B47C09',
'226147F7-68D1-45FF-ACDE-DDD032870FB2',
'71335D4E-34B0-4FE4-B2C7-C00585A54285',
'A9AF39E5-0BD6-41A7-B0C6-A9DC4CB85FD7',
'1F1FED5A-68EF-4A32-9A15-84E3646440C5',
'C6EA36EA-B74D-499A-AC1F-84024CD55CB4',
'21E90526-2978-4653-BF8C-804D67F7B641',
'7710B99B-2BDE-4BF6-BB21-73674F531C50',
'266711DC-5643-4D57-8F2E-7347B8EFF521',
'C9B583C2-4043-44AB-992E-6FAD0523FFED',
'B5BC2EB1-0A30-4391-8C16-6E239DF08C85',
'BFD476A2-42D9-4596-87DA-59E18EB27ED7',
'CA568921-3856-4409-8446-522F01DC8598',
'5CC8FB4A-C912-4302-94EA-34A577A54BF7',
'71E9CD51-A1E0-485A-BD48-33FB49458943',
'C4EDB622-B9B7-414B-9BD9-30FF20C38A2F',
'8B04D80B-925B-4A7C-AF8C-1B6BB2DBC861',
'C405D8A1-7503-4CA9-B292-1A90CB2561F9',
'AA4B66D3-C2BB-444B-A003-05D334344DF2'
);"""
cursor.execute(query)
# Fetch results
conversation_ids = cursor.fetchall()
conversation_ids = [row for row in conversation_ids if row[0] != '']
logging.info('Print Conversation_ids-------------------.')
print("Print Conversation_ids-------------------")
#logging.info(conversation_ids)
#for row in conversation_ids:
# print(row.ConversationId)
# Close connection
cursor.close()
conn.close()
OUTLINE
TIMELINE
Ln 70, Col 27 Spaces: 4
Search


File Edit Selection View Go Run Terminal Help
mtr-vdi-cp62
Ctrl+Alt+Del
USB Devices
Exit Fullscreen
EXPLORER
function_app.py
local.settings.json
GENESYSCONVERSATIONS
.venv
.vscode
.funcignore
.gitignore
function_app.py
host.json
local.settings.json
requirements.txt
function_app.py > my_function
def my_function(custom_flag, custom_date):
cursor.close()
conn.close()
except Exception as e:
print("Error while connecting to SQL Server:", e)
logging.error("An error occurred: %s", e, exc_info=True)
Run Cell | Run Above | Debug Cell
# %%
# Azure storage account details
account_url = "https://ogeprodmeteranalytics.blob.core.windows.net"
container_name = "genesys-api"
blob_name = "conversations"  # path inside container
credential = "PQSzkVWt0N5+aCuKg+Vq2O9LLSCT/Hbqm1jrgFGg8T319D4P0XQO3FW1GUuItKIPYEG+yozARWS2+ASteYqimA=="
# Create Blob service client
blob_service_client = BlobServiceClient(account_url=account_url, credential=credential)
# Get container client
container_client = blob_service_client.get_container_client(container_name)
Run Cell | Run Above | Debug Cell
# %%
if custom_flag == "N":
yesterday = (datetime.now() - timedelta(days=1)).strftime("%Y-%m-%d")
elif custom_flag == "Y":
yesterday = custom_date
blob_name = f"conversations/{yesterday}/.placeholder"
# Get a blob client for the new "folder"
blob_client = blob_service_client.get_blob_client(container=container_name, blob=blob_name)
# Upload an empty content to create the virtual folder
blob_client.upload_blob(b"", overwrite=True)
logging.info("uploading the empty file to blob")
Run Cell | Run Above
# %% [markdown]
# **Step 4:** Update the access token and execute the script. Script will pause after each request to avoid hitting api rate limitations.
Run Cell | Run Above | Debug Cell
# %%
OUTLINE
TIMELINE
Ln 70, Col 27
Spaces: 4
UTF-8
CRLF
Search

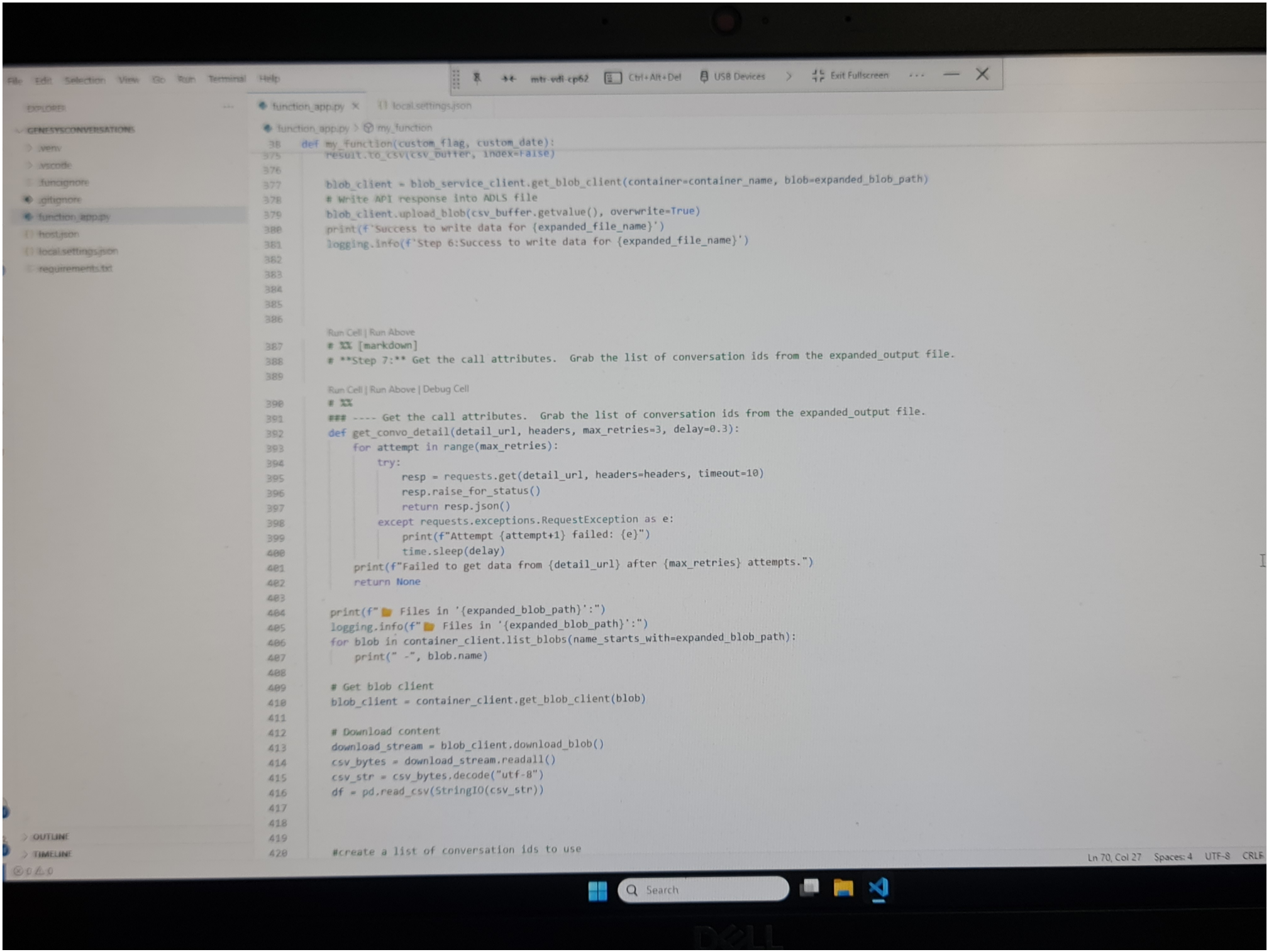

File Edit Selection View Go Run Terminal Help
mtr-vdi-cp62
Ctrl+Alt+Del
USB Devices
Exit Fullscreen
EXPLORER
GENESYSCONVERSATIONS
.venv
.vscode
.funcignore
.gitignore
function_app.py
host.json
local.settings.json
requirements.txt
function_app.py
local.settings.json
function_app.py > my_function
38 def my_function(custom_flag, custom_date):
201 def get_summary(url, headers, max_retries=3, delay=0.3):
202 for attempt in range(max_retries):
203 try:
204 response = requests.get(url, headers=headers, timeout=10)
205
206 if response.status_code == 200:
207 return response
208 elif response.status_code == 404:
209 # do not retry this record
210 print(f"Attempt {attempt+1}: Skipping {url} - Not Found")
211
212 except requests.exceptions.RequestException as e:
213 print(f"Attempt {attempt+1}: Exception for {url} - {e}")
214 time.sleep(delay)
215 print(f"Giving up on {url} after {max_retries} attempts.")
216
217
218 headers = {
219 'Authorization': f'Bearer {access_token}',
220 'Content-Type': 'application/json'
221 }
222
223 file_path = f"conversations/{yesterday}/"
224 record_count = 1
225
226 #for conversation_id in conversation_ids:
227 for row in conversation_ids:
228 conversation_id =row.ConversationId
229 print(conversation_id)
230 url = f'https://api.usw2.pure.cloud/api/v2/conversations/{conversation_id}/summaries'
231
232 response = get_summary(url, headers)
233
234
235 if response is not None:
236 print("Response:")
237 print(response)
238 file_name = f'{conversation_id}.json'
239 blob_path = file_path+file_name
240 blob_client = blob_service_client.get_blob_client(container=container_name, blob=blob_path)
241 # Write API response into ADLS file
242 blob_client.upload_blob(response, overwrite=True)
243 print(f'Success to get data for {conversation_id}: {response.status_code}')
244 print(f'{record_count} of {len(conversation_ids)}. Saved response to {file_path+file_name}')
245 else:
246 print(f'Failed to get data for {conversation_id}')
247 print(f'{record count} of {len(conversation ids)}. Skipped {conversation id} due to repeated failures.')
OUTLINE
TIMELINE
Ln 70, Col 27 Spaces: 4 UTF-8 CRL
Search

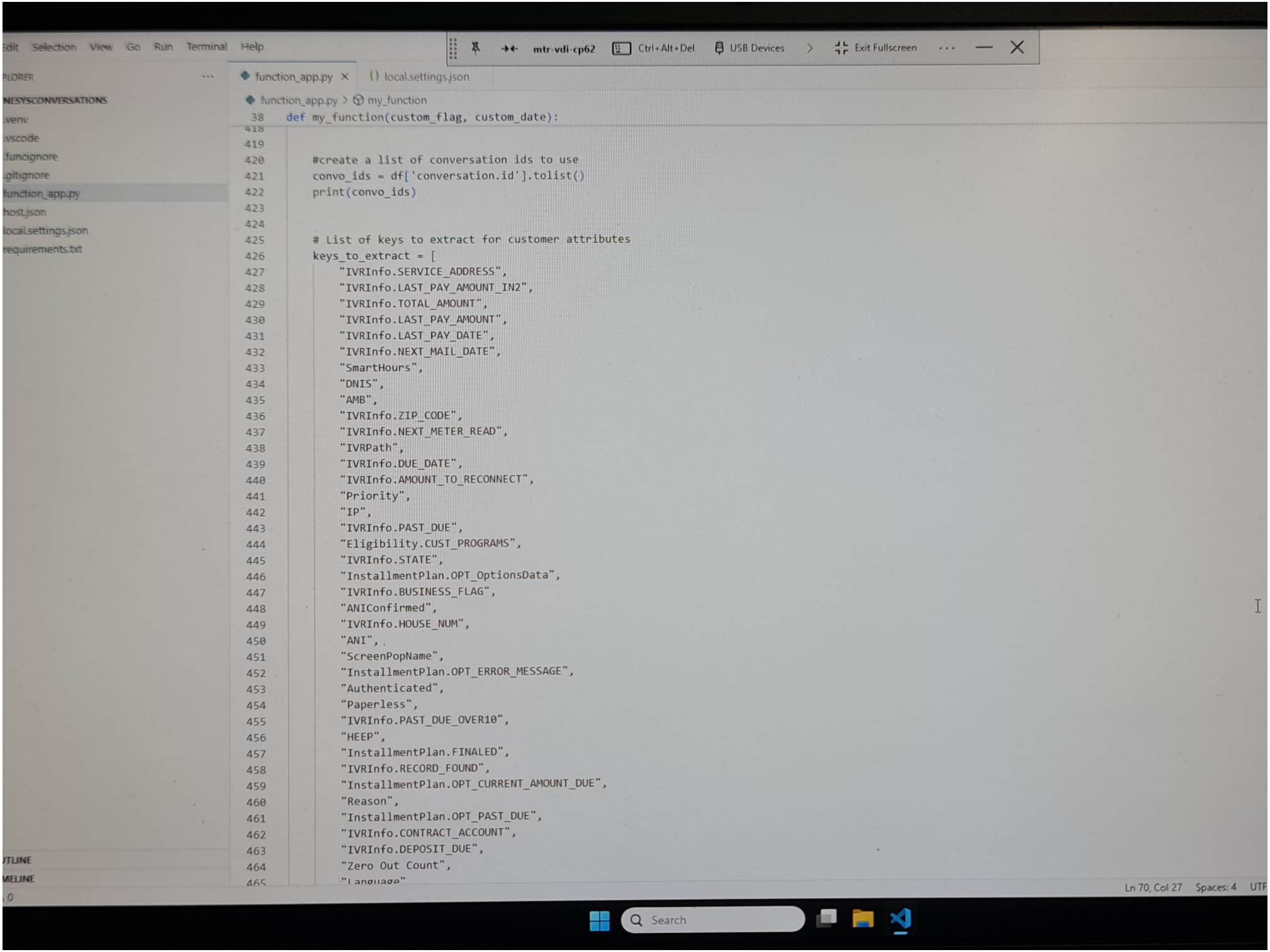

File Edit Selection View Go Run Terminal Help
mtr-vdi-cp62
Ctrl+Alt+Del
USB Devices
Exit Fullscreen
EXPLORER
GENESYSCONVERSATIONS
.venv
.vscode
.funcignore
.gitignore
function_app.py
host.json
local.settings.json
requirements.txt
function_app.py
local.settings.json
function_app.py > my_function
def my_function(custom_flag, custom_date):
else:
print(f'Failed to get data for {conversation_id}')
print(f'{record_count} of {len(conversation_ids)}. Skipped {conversation_id} due to repeated failures.')
time.sleep(0.3) # Pause for 0.3 seconds to avoid hitting rate limits
record_count += 1
Run Cell | Run Above | Debug Cell
# %%
logging.info(f"📂 Files in '{file_path}':")
for blob in container_client.list_blobs(name_starts_with=file_path):
print(" -", blob.name)
logging.info(blob.name)
Run Cell | Run Above
# %% [markdown]
# **Step 5:** Combine json responses into a single csv file.
Run Cell | Run Above | Debug Cell
# %%
### --- Combine json responses into a single csv file.
# List to hold data from all files
data = []
# Loop through all JSON files in the directory
#for filename in os.listdir(json_dir):
for blob in container_client.list_blobs(name_starts_with=file_path):
if blob.name.endswith('.json'):
print(f"📄 Processing blob: {blob.name}")
# Get blob client
blob_client = container_client.get_blob_client(blob)
# Download content
content = blob_client.download_blob().readall()
# If JSON, load into variable
try:
file_data = json.loads(content)
print("✅ JSON content loaded:", blob_path)
except json.JSONDecodeError:
print("⚠️ Not a JSON file, raw content length:", len(content))
if isinstance(file_data, list):
OUTLINE
TIMELINE
Ln 70, Col 27
Spaces: 4
UTF-8
CRL
Search


Selection View Go Run Terminal Help
mtr-vdi-cp62 Ctrl+Alt+Del USB Devices Exit Fullscreen
function_app.py
local.settings.json
function_app.py > my_function
def my_function(custom_flag, custom_date):
print(" Not a JSON file, raw content length:", len(content))
if isinstance(file_data, list):
data.extend(file_data)
else:
data.append(file_data)
# Use json_normalize to expand nested fields
df = pd.json_normalize(data)
#print(" normalized data ", df)
# Output to CSV
combined_file_name = f'{yesterday}.csv'
combined_file_path = 'conversations_combined/'
combined_blob_path = combined_file_path+combined_file_name
csv_buffer = StringIO()
df.to_csv(csv_buffer, index=False)
blob_client = blob_service_client.get_blob_client(container=container_name, blob=combined_blob_path)
# Write API response into ADLS file
blob_client.upload_blob(csv_buffer.getvalue(), overwrite=True)
print(f'Success to write data for {combined_file_name}')
Run Cell | Run Above
# %% [markdown]
# **Step 6:** Expand all json responses and double-quote each csv column.
Run Cell | Run Above | Debug Cell
# %%
### ----- Expand all json responses and double-quote each csv column.
print(f" Files in '{combined_blob_path}':")
for blob in container_client.list_blobs(name_starts_with=combined_blob_path):
print(" -", blob.name)
# Get blob client
blob_client = container_client.get_blob_client(blob)
# Download content
download_stream = blob_client.download_blob()
csv_bytes = download_stream.readall()
csv_str = csv_bytes.decode("utf-8")
df = pd.read_csv(StringIO(csv_str))
Ln 70, Col 27 Spaces: 4
Search



```
def my_function(custom_flag, custom_date):
    csv_str = csv_bytes.decode("utf-8")
    df = pd.read_csv(StringIO(csv_str))


    # Parse the sessionSummaries column (string to list of dicts)
    def parse_session_summaries(val):
        try:
            return ast.literal_eval(val)
        except Exception:
            return []

    # Expand the first session summary in each row
    expanded = df['sessionSummaries'].apply(parse_session_summaries).apply(lambda x: x[0] if x else {})

    # Create a DataFrame from the expanded session summaries
    expanded_df = pd.json_normalize(expanded)

    # Combine with the other columns (excluding the original sessionSummaries)
    result = pd.concat([expanded_df, df.drop(columns=['sessionSummaries'])], axis=1)

    # remove data from the column editedSummary.text to avoid exposing any PII
    result['editedSummary.text'] = ''

    # drop any row with an error type of 'ConversationTooShort'
    try:
        result = result[result['errorType'] != 'ConversationTooShort']
    except Exception as e:
        print(e)
        logging.error("An error occurred: %s", e, exc_info=True)


    # remove line breaks for the text column
    result['text'] = result['text'].str.replace('\n', ' ')

    # remove quote characters
    result['text'] = result['text'].str.replace('"', '')


    # Output to CSV
    expanded_file_name = f'{yesterday}.csv'
    expanded_file_path = 'conversations_expanded/'
    expanded_blob_path = expanded_file_path+expanded_file_name

    csv_buffer = StringIO()
    result.to_csv(csv_buffer, index=False)

    blob_client = blob_service_client.get_blob_client(container=container_name, blob=expanded_blob_path)
```


File Edit Selection View Go Run Terminal Help
mtr-vdi-cp62
Ctrl+Alt+Del
USB Devices
Exit Fullscreen
EXPLORER
GENESYSCONVERSATIONS
.venv
.vscode
.funcignore
.gitignore
function_app.py
host.json
local.settings.json
requirements.txt
function_app.py
local.settings.json
function_app.py > my_function
def my_function(custom_flag, custom_date):
result.to_csv(csv_buffer, index=False)
blob_client = blob_service_client.get_blob_client(container=container_name, blob=expanded_blob_path)
# Write API response into ADLS file
blob_client.upload_blob(csv_buffer.getvalue(), overwrite=True)
print(f'Success to write data for {expanded_file_name}')
logging.info(f'Step 6:Success to write data for {expanded_file_name}')
Run Cell | Run Above
# %% [markdown]
# **Step 7:** Get the call attributes. Grab the list of conversation ids from the expanded_output file.
Run Cell | Run Above | Debug Cell
# %%
### ---- Get the call attributes. Grab the list of conversation ids from the expanded_output file.
def get_convo_detail(detail_url, headers, max_retries=3, delay=0.3):
for attempt in range(max_retries):
try:
resp = requests.get(detail_url, headers=headers, timeout=10)
resp.raise_for_status()
return resp.json()
except requests.exceptions.RequestException as e:
print(f"Attempt {attempt+1} failed: {e}")
time.sleep(delay)
print(f"Failed to get data from {detail_url} after {max_retries} attempts.")
return None
print(f"📂 Files in '{expanded_blob_path}':")
logging.info(f"📂 Files in '{expanded_blob_path}':")
for blob in container_client.list_blobs(name_starts_with=expanded_blob_path):
print(" -", blob.name)
# Get blob client
blob_client = container_client.get_blob_client(blob)
# Download content
download_stream = blob_client.download_blob()
csv_bytes = download_stream.readall()
csv_str = csv_bytes.decode("utf-8")
df = pd.read_csv(StringIO(csv_str))
#create a list of conversation ids to use
OUTLINE
TIMELINE
Ln 70, Col 27 Spaces: 4 UTF-8 CRLF
Search


Edit Selection View Go Run Terminal Help
mtr-vdi-cp62 Ctrl+Alt+Del USB Devices Exit Fullscreen
PLORER
NESYSCONVERSATIONS
.venv
.vscode
.funcignore
.gitignore
function_app.py
host.json
local.settings.json
requirements.txt
function_app.py
local.settings.json
function_app.py > my_function
def my_function(custom_flag, custom_date):
#create a list of conversation ids to use
convo_ids = df['conversation.id'].tolist()
print(convo_ids)
# List of keys to extract for customer attributes
keys_to_extract = [
"IVRInfo.SERVICE_ADDRESS",
"IVRInfo.LAST_PAY_AMOUNT_IN2",
"IVRInfo.TOTAL_AMOUNT",
"IVRInfo.LAST_PAY_AMOUNT",
"IVRInfo.LAST_PAY_DATE",
"IVRInfo.NEXT_MAIL_DATE",
"SmartHours",
"DNIS",
"AMB",
"IVRInfo.ZIP_CODE",
"IVRInfo.NEXT_METER_READ",
"IVRPath",
"IVRInfo.DUE_DATE",
"IVRInfo.AMOUNT_TO_RECONNECT",
"Priority",
"IP",
"IVRInfo.PAST_DUE",
"Eligibility.CUST_PROGRAMS",
"IVRInfo.STATE",
"InstallmentPlan.OPT_OptionsData",
"IVRInfo.BUSINESS_FLAG",
"ANIConfirmed",
"IVRInfo.HOUSE_NUM",
"ANI",
"ScreenPopName",
"InstallmentPlan.OPT_ERROR_MESSAGE",
"Authenticated",
"Paperless",
"IVRInfo.PAST_DUE_OVER10",
"HEEP",
"InstallmentPlan.FINALED",
"IVRInfo.RECORD_FOUND",
"InstallmentPlan.OPT_CURRENT_AMOUNT_DUE",
"Reason",
"InstallmentPlan.OPT_PAST_DUE",
"IVRInfo.CONTRACT_ACCOUNT",
"IVRInfo.DEPOSIT_DUE",
"Zero Out Count",
"Language"
Ln 70, Col 27 Spaces: 4 UTF
Search

```
def my_function(custom_flag, custom_date):
        "IVRInfo.BUSINESS_PARTNER",
        "Last4Check.RETURN_CODE"
    ]



    # Set up the headers, including the authorization token
    headers = {
        'Authorization': f'Bearer {access_token}',
        'Content-Type': 'application/json'
    }

    # place the ids into batches of 50
    batches = [convo_ids[i:i + 50] for i in range(0, len(convo_ids), 50)]

    # Set to keep track of unique conversation IDs
    unique_conversations = set()

    output = StringIO()
    output_obj = []
    header = ["conversation_id"] + keys_to_extract
    for batch in batches:
        #pause for 1 second between batches
        time.sleep(0.3)
        # print the number of the batch being processed
        print(f"Processing batch {batches.index(batch) + 1} of {len(batches)} : {batch}")

    #loop through the convo_ids
        for conversation_id in batch:
            print(conversation_id)
            # sleep for 1/2 seconds to avoid hitting API rate limits
            time.sleep(0.3)
            # ---- Get full conversation detail ----
            detail_url = f"https://api.{region}.pure.cloud/api/v2/conversations/{conversation_id}"
            #detail_resp = requests.get(detail_url, headers=headers)
            #convo_detail = detail_resp.json()
            convo_detail = get_convo_detail(detail_url, headers)

            if convo_detail is None:
                continue  # skip this conversation_id

            customer_attrs = {}
            for part in convo_detail.get("participants", []):
                purpose = part.get("purpose")
                if purpose == "customer":
                    customer_attrs = part.get("attributes", {})
```

View Go Run Terminal Help

mtr-vdi-cp62 Ctrl+Alt+Del USB Devices Exit Fullscreen

function_app.py local.settings.json

function_app.py > my_function

```
38  def my_function(custom_flag, custom_date):
529                 if purpose == "customer":
530                     customer_attrs = part.get("attributes", {})
531                     # Extracting the specified keys
532                     # if a key is not present in the customer_attrs, it will be skipped
533                     extracted_data = {key: customer_attrs[key] for key in keys_to_extract if key in customer_attrs}
534
535                     row = [conversation_id]
536                     row_temp = []
537                     for col_name in header:
538                         val = str(extracted_data.get(col_name, conversation_id if col_name == "conversation_id" else "")).replace("\n", " ").replace("\r", " ")
539                         row_temp.append(val)
540
541                     # if the length of the line is greater than 250 characters, take it, otherwise skip it
542                     if len(",".join(row)) > 250:
543                         row.append(extracted_data.get(key, ""))
544
545                     for key in keys_to_extract:
546                         row.append(extracted_data.get(key, ""))
547
548                     # Write header if it's the first row
549                     if len(unique_conversations) == 0:
550                         writer = csv.writer(output, quoting=csv.QUOTE_ALL)
551                         writer.writerow(header)
552                         output_obj.append(header)
553                     else:
554                         writer = csv.writer(output, quoting=csv.QUOTE_ALL)
555                     # Write the row as CSV with all columns quoted
556                     writer.writerow([str(item).replace('\n', ' ').replace('\r', ' ') for item in row])
557                     # display the row
558                     #print(row)
559                     output_obj.append(row)
560                     #@output_obj is a list object
561                     # Add the conversation ID to the set of unique conversations
562                     unique_conversations.add(conversation_id)
563
564
565
566
567     # remove any blank rows for the csv file
568     lines = [item for item in output_obj if item not in ("", None, [])]
569     lines = [item for item in lines if any(cell.strip() for cell in row)]
570     # if the length is less than 250 delete the row
571     #lines = [item for item in lines if len(item) >= 250]
572
573     lines = [
574         item for item in lines
575         if any(col not in (None, '') for col in item[1:])  # check columns after first
```

Ln 70, Col 27 Spaces: 4 UTF-8 CRLF Pyth

Search

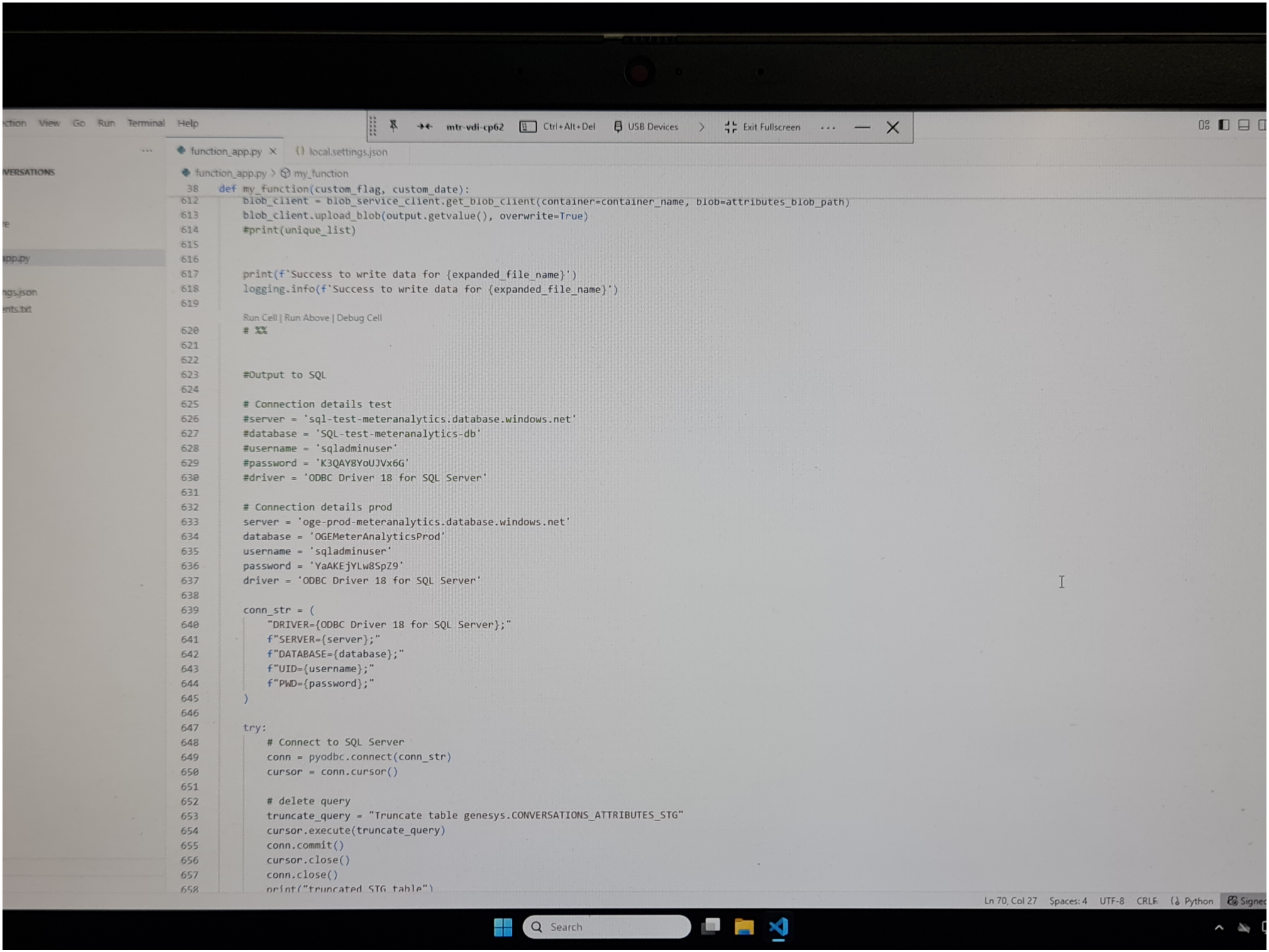
Terminal Help

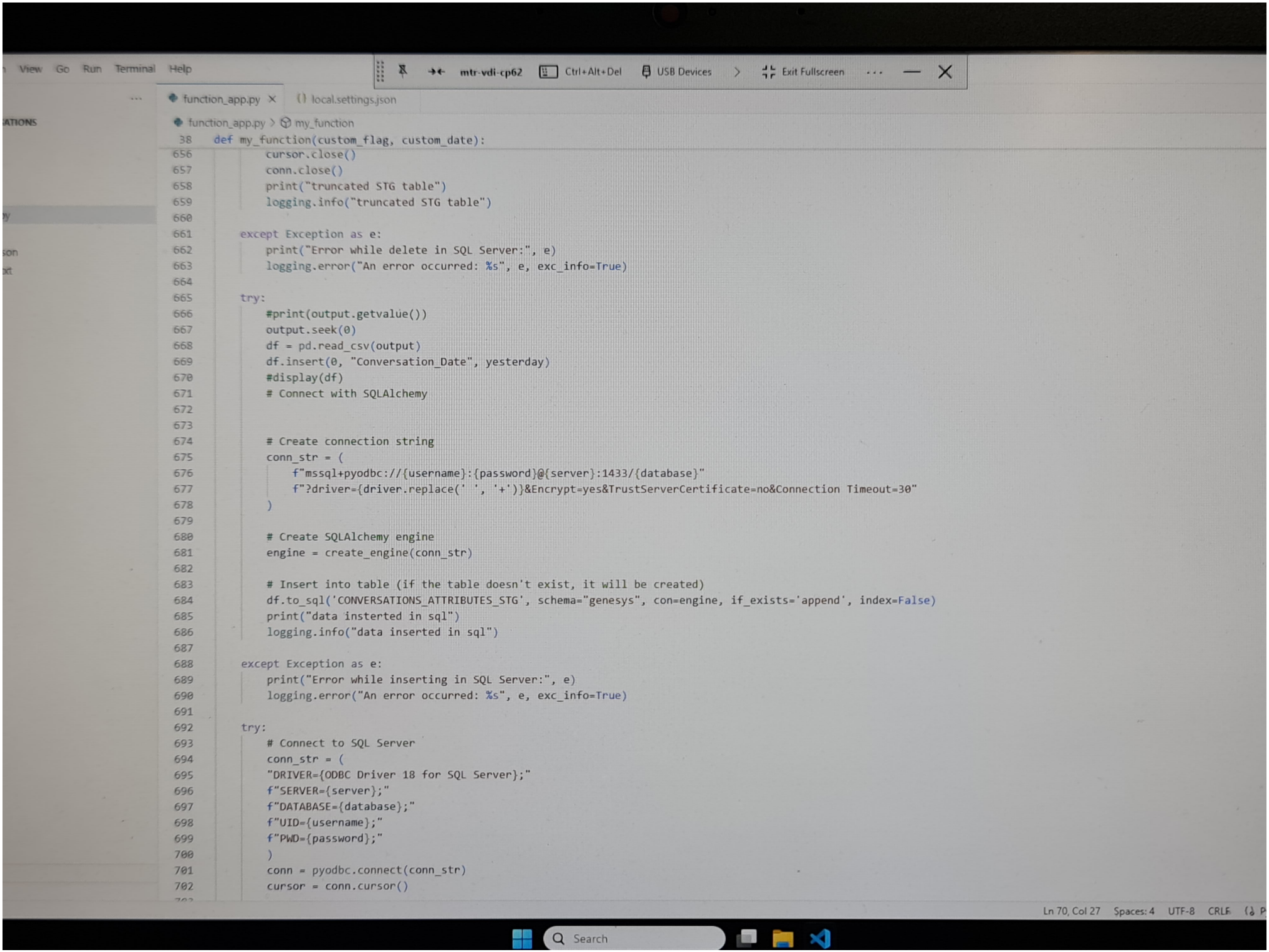
mtr-vdi-cp62 Ctrl+Alt+Del USB Devices Exit Fullscreen

function_app.py local.settings.json

function_app.py > my_function

```
38  def my_function(custom_flag, custom_date):
571     #lines = [item for item in lines if len(item) >= 250]
572
573     lines = [
574         item for item in lines
575         if any(col not in (None, '') for col in item[1:])  # check columns after first
576     ]
577
578     # test = []
579     # for item in lines:
580     #     for col in item[1:]
581
582     print(lines)
583
584
585
586     # Remove any duplicate rows
587     seen = set()
588     unique_list = []
589     for row in lines:
590         row_tuple = tuple(row)  # convert list to tuple (hashable)
591         if row_tuple not in seen:
592             seen.add(row_tuple)
593             unique_list.append(row)
594
595
596
597
        Run Cell | Run Above | Debug Cell
598     # %%
599
600
601     # Output to CSV
602     attributes_file_name = f'{yesterday}.csv'
603     attributes_file_path = 'conversations_attributes/'
604     attributes_blob_path = attributes_file_path+attributes_file_name
605
606     output = io.StringIO()
607     writer = csv.writer(output, quoting=csv.QUOTE_ALL)
608     for row in unique_list:
609         writer.writerow(row)
610
611
612     blob_client = blob_service_client.get_blob_client(container=container_name, blob=attributes_blob_path)
613     blob_client.upload_blob(output.getvalue(), overwrite=True)
614     #print(unique_list)
615
616
```


mtr-vdi-cp62
Ctrl+Alt+Del
USB Devices
Exit Fullscreen
function_app.py
local.settings.json
function_app.py > my_function
def my_function(custom_flag, custom_date):
blob_client = blob_service_client.get_blob_client(container=container_name, blob=attributes_blob_path)
blob_client.upload_blob(output.getvalue(), overwrite=True)
#print(unique_list)
print(f'Success to write data for {expanded_file_name}')
logging.info(f'Success to write data for {expanded_file_name}')
Run Cell | Run Above | Debug Cell
# %%
#Output to SQL
# Connection details test
#server = 'sql-test-meteranalytics.database.windows.net'
#database = 'SQL-test-meteranalytics-db'
#username = 'sqladminuser'
#password = 'K3QAY8YoUJVx6G'
#driver = 'ODBC Driver 18 for SQL Server'
# Connection details prod
server = 'oge-prod-meteranalytics.database.windows.net'
database = 'OGEMeterAnalyticsProd'
username = 'sqladminuser'
password = 'YaAKEjYLw8SpZ9'
driver = 'ODBC Driver 18 for SQL Server'
conn_str = (
"DRIVER={ODBC Driver 18 for SQL Server};"
f"SERVER={server};"
f"DATABASE={database};"
f"UID={username};"
f"PWD={password};"
)
try:
# Connect to SQL Server
conn = pyodbc.connect(conn_str)
cursor = conn.cursor()
# delete query
truncate_query = "Truncate table genesys.CONVERSATIONS_ATTRIBUTES_STG"
cursor.execute(truncate_query)
conn.commit()
cursor.close()
conn.close()
print("truncated STG table")
Ln 70, Col 27
Spaces: 4
UTF-8
CRLF
Python
Search


View Go Run Terminal Help
mtr-vdi-cp62 Ctrl+Alt+Del USB Devices Exit Fullscreen
function_app.py
local.settings.json
function_app.py > my_function
38 def my_function(custom_flag, custom_date):
656 cursor.close()
657 conn.close()
658 print("truncated STG table")
659 logging.info("truncated STG table")
660
661 except Exception as e:
662 print("Error while delete in SQL Server:", e)
663 logging.error("An error occurred: %s", e, exc_info=True)
664
665 try:
666 #print(output.getvalue())
667 output.seek(0)
668 df = pd.read_csv(output)
669 df.insert(0, "Conversation_Date", yesterday)
670 #display(df)
671 # Connect with SQLAlchemy
672
673
674 # Create connection string
675 conn_str = (
676 f"mssql+pyodbc://{username}:{password}@{server}:1433/{database}"
677 f"?driver={driver.replace(' ', '+')}&Encrypt=yes&TrustServerCertificate=no&Connection Timeout=30"
678 )
679
680 # Create SQLAlchemy engine
681 engine = create_engine(conn_str)
682
683 # Insert into table (if the table doesn't exist, it will be created)
684 df.to_sql('CONVERSATIONS_ATTRIBUTES_STG', schema="genesys", con=engine, if_exists='append', index=False)
685 print("data insterted in sql")
686 logging.info("data inserted in sql")
687
688 except Exception as e:
689 print("Error while inserting in SQL Server:", e)
690 logging.error("An error occurred: %s", e, exc_info=True)
691
692 try:
693 # Connect to SQL Server
694 conn_str = (
695 "DRIVER={ODBC Driver 18 for SQL Server};"
696 f"SERVER={server};"
697 f"DATABASE={database};"
698 f"UID={username};"
699 f"PWD={password};"
700 )
701 conn = pyodbc.connect(conn_str)
702 cursor = conn.cursor()
Ln 70, Col 27 Spaces: 4 UTF-8 CRLF
Search